Synchronous concurrent refinement algebra and justness

Ian J. Hayes
The University of Queensland, Australia
work with Robert J. Colvin, Larissa A. Meinicke, Kirsten Winter (UQ)
Cliff Jones, Andrius Velykis (Newcastle University)

IFIP WG2.3 – Chateau Villebrumier – 4 October 2016

Overview

- Brief overview of synchronous concurrent refinement algebra
- Justness – a form of fairness

Structure of concurrent program algebra

- Concurrent refinement algebra ($\sqcap$, $\sqcup$, abort, magic, ; , nil, $\|$, skip, $\Cap$, chaos)
- Plus tests – a subset of commands that forms a boolean algebra (like Kozen's KAT)
- Plus atomic steps – a subset of commands that forms a boolean algebra
- Program/environment steps – partitions atomic steps
- Relational instantiation

Aczel traces

Every step of parallel synchronises steps of the two processes

$$\begin{array}{ll} \mathcal{E}(\sigma_0,\sigma_1),\Pi(\sigma_1,\sigma_2),\mathcal{E}(\sigma_2,\sigma_3),\mathcal{E}(\sigma_3,\sigma_4),\mathcal{E}(\sigma_4,\sigma_5) & \| \\ \mathcal{E}(\sigma_0,\sigma_1),\mathcal{E}(\sigma_1,\sigma_2),\Pi(\sigma_2,\sigma_3),\mathcal{E}(\sigma_3,\sigma_4),\Pi(\sigma_4,\sigma_5) & = \\ \mathcal{E}(\sigma_0,\sigma_1),\Pi(\sigma_1,\sigma_2),\Pi(\sigma_2,\sigma_3),\mathcal{E}(\sigma_3,\sigma_4),\Pi(\sigma_4,\sigma_5) & \end{array}$$

Some primitive commands

For a binary relation $r \subseteq \Sigma \times \Sigma$ on states

$\pi(r)$ can perform any single program step $\Pi(\sigma, \sigma')$ for $(\sigma, \sigma') \in r$

$\epsilon(r)$ can perform any single environment step $\mathcal{E}(\sigma, \sigma')$ for $(\sigma, \sigma') \in r$

For example,

- $\boldsymbol{\pi}(\mathrm{id})$ is a single stuttering program step
- $\boldsymbol{\pi} \mathrel{\widehat{=}} \boldsymbol{\pi}(\mathrm{univ})$ can perform any single program step
- $\boldsymbol{\epsilon} \mathrel{\widehat{=}} \boldsymbol{\epsilon}(\mathrm{univ})$ can perform any single environment step
- $\boldsymbol{\pi}(\varnothing) = \boldsymbol{\epsilon}(\varnothing) =$ magic is infeasible

Operators

$c \sqcap d$ non-deterministic choice (lattice meet)

$c \sqcup d$ lattice join

$c \,;\, d$ sequential composition (elided to $c\,d$ below)

$c \parallel d$ parallel composition

$c \Cap d$ weak conjunction

Combining primitive commands with parallel and weak conjunction

$$
\begin{aligned}
\pi(r_1) \parallel \pi(r_2) &= \text{magic} \\
\pi(r_1) \parallel \epsilon(r_2) &= \pi(r_1 \cap r_2) \\
\epsilon(r_1) \parallel \epsilon(r_2) &= \epsilon(r_1 \cap r_2) \\
\pi(r) \parallel \text{abort} &= \text{abort} \\
\epsilon(r) \parallel \text{abort} &= \text{abort}
\end{aligned}
$$

$$
\begin{aligned}
\pi(r_1) \Cap \pi(r_2) &= \pi(r_1 \cap r_2) \\
\pi(r_1) \Cap \epsilon(r_2) &= \text{magic} \\
\epsilon(r_1) \Cap \epsilon(r_2) &= \epsilon(r_1 \cap r_2) \\
\pi(r) \Cap \text{abort} &= \text{abort} \\
\epsilon(r) \Cap \text{abort} &= \text{abort}
\end{aligned}
$$

Iteration

Iteration zero or more times, c^{ω}, allows finite iteration, $c^{\star}$, or infinite iteration, c^{∞}

$$c^{\omega} = c^{\star} \sqcap c^{\infty} \tag{1}$$

Examples

$\pi^{\star}$ can perform a finite number of program steps

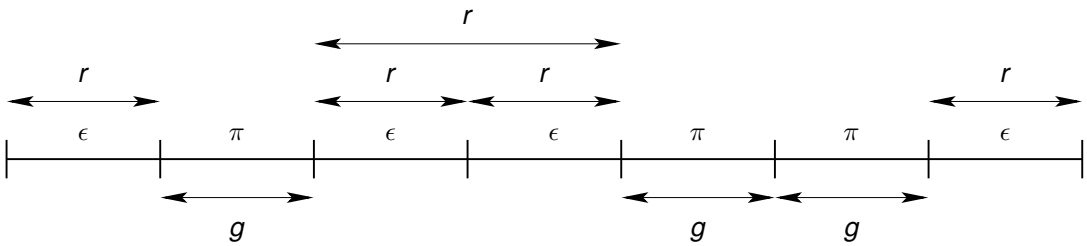
$(\pi \sqcap \epsilon)^{\star}$ can perform a finite number of steps

ϵ^{∞} performs an infinite sequence of environment steps

skip is the identity of parallel and chaos is the identity of weak conjunction

$$\begin{aligned} \text{skip} &\mathrel{\widehat{=}} \epsilon^{\omega} \\ \text{chaos} &\mathrel{\widehat{=}} (\pi \sqcap \epsilon)^{\omega} \end{aligned}$$

Rely/guarantee



Guarantee and rely

For relations g and r

$$\begin{aligned} \text{guar } g &\mathrel{\widehat{=}} (\pi(g) \sqcap \epsilon)^\omega \\ \text{demand } r &\mathrel{\widehat{=}} (\pi \sqcap \epsilon(r))^\omega \\ \text{rely } r &\mathrel{\widehat{=}} (\text{demand } r)\,(\text{nil} \sqcap \epsilon(\bar{r})\,\text{abort}) \end{aligned}$$

For example, $c \Cap (\text{guar } g) \Cap (\text{rely } r)$ imposes a guarantee of g on c and assumes the environment steps satisfy r.

Rely-guarantee specification of a component

- s is a set of natural numbers
- $mults(i)$ is the set of all multiples of i (excluding i)
- Specification

$$(\text{guar}\ \ s' \subseteq s \land s - s' \subseteq mults(i)) \Cap (\text{rely}\ \ s' \subseteq s) \Cap \{s = 2\,..\,N\}\,[s' \cap mults(i) = \varnothing]$$
$$\sqsubseteq$$
$$rem\_mult(i)$$

Termination

The command term allows only a finite number of program steps but does not rule out infinite pre-emption by its environment.

$$\text{term} \mathrel{\widehat{=}} \epsilon^{\omega}\,(\pi\,\epsilon^{\omega})^{\star} \tag{2}$$

The refinement

$$\text{term} \sqsubseteq c$$

states that c terminates if the environment does not interrupt it forever, e.g.

$$\text{term} \sqsubseteq x := 1$$

Imposing justness (a form of fairness)

The process just allows any behaviour, except pre-emption by its environment forever,

$$\text{just} \mathrel{\widehat{=}} \boldsymbol{\epsilon}^\star \, (\boldsymbol{\pi} \, \boldsymbol{\epsilon}^\star)^\omega \tag{3}$$

It requires all contiguous subsequences of environment steps to be finite.

Just termination

$$\begin{aligned} \mathsf{term} \mathbin{⫛} \mathsf{just} &= \epsilon^{\omega} \, (\pi \, \epsilon^{\omega})^{\star} \mathbin{⫛} \epsilon^{\star} \, (\pi \, \epsilon^{\star})^{\omega} \\ &= \epsilon^{\star} \, (\pi \, \epsilon^{\star})^{\star} \\ &= (\pi \sqcap \epsilon)^{\star} \end{aligned}$$

Hence if

$$\mathsf{term} \sqsubseteq c$$

then

$$\mathsf{term} \mathbin{⫛} \mathsf{just} \sqsubseteq c \mathbin{⫛} \mathsf{just}$$

i.e. just-execution of c gives strong termination.

Just execution

A process representing just-execution of a process c is represented by

$$c \Cap \mathrm{just}$$

Because just never aborts, any aborting behaviour of $c \Cap \mathrm{just}$ arises solely from c.

... just execution

Execution of the process,

$$\textbf{do}\ \text{true} \rightarrow y := y + 1\ \textbf{od} \tag{4}$$

with y initially zero may never set y to 7.
In contrast,

$$\textbf{do}\ \text{true} \rightarrow y := y + 1\ \textbf{od}\ \Cap\ \textbf{just} \tag{5}$$

rules out infinite pre-emption by its environment and hence ensures that eventually y becomes 7 (or any other natural number).

Unjust parallel

For a parallel composition, $c \parallel d$, the execution of c may be pre-empted forever by the execution of d, or vice versa.

$$x := 1 \parallel \text{do}\, x \neq 1 \rightarrow y := y + 1\, \text{od} \tag{6}$$

Just parallel

$$((x := 1) \Cap \text{just}) \parallel_e (\text{do } x \neq 1 \rightarrow y := y + 1 \text{ od} \Cap \text{just}) \tag{7}$$

The interpretation of $\parallel_e$ is that termination of one process of a parallel composition reduces the composition to the other process, i.e. $\text{nil} \parallel_e c = c$. One may define just-parallel as follows.

$$c \parallel_j d \;\widehat{=}\; (c \Cap \text{just}) \parallel_e (d \Cap \text{just}) \tag{8}$$

Note that

$$(c \parallel_e d) \Cap \text{just} \;\sqsubseteq\; c \parallel_j d \tag{9}$$

but the reverse refinement doesn't hold in general because $(c \parallel_e d) \Cap \text{just}$ doesn't rule out c being pre-empted forever by d (or vice versa) within the parallel.

Parallel with synchronised termination

For parallel with synchronised termination one only has

$$\mathsf{nil} \parallel \mathsf{nil} = \mathsf{nil}$$

but not the more general property of parallel with early termination.

$$\mathsf{nil} \parallel_e c = c$$

For parallel with synchronised termination,

$$(x := 1 \Cap \mathsf{just}) \parallel (\mathsf{do}\,\mathsf{true} \to y := y + 1\,\mathsf{od} \Cap \mathsf{just}) \tag{10}$$

is infeasible.

$$c \parallel_e d \;\widehat{=}\; (c\,\mathsf{skip}) \parallel (d\,\mathsf{skip}) \tag{11}$$

The definition of just-parallel then expands as follows.

$$c \parallel_j d \widehat{=} ((c \Cap \mathsf{just})\,\mathsf{skip}) \parallel ((d \Cap \mathsf{just})\,\mathsf{skip}) \tag{12}$$

That imposes justness on c until it terminates, and similarly for d.

Applying to process algebras

The above approach can also be applied to CSP-style processes.

- $\Pi(a)$ is interpreted as an atomic event or action a
- $\mathcal{E}(a)$ is a corresponding environment event
- $\pi(A)$ allows any event $\Pi(a)$ for any $a \in A$
- $\epsilon(A)$ allows any environment event $\mathcal{E}(a)$ for any $a \in A$.
- $\pi(A) \parallel \pi(B) = \pi(A \cap B)$

Summary

- Aczel's parallel operator is synchronous, similar to Milner's SCCS
- Using synchronous atomic steps algebra one can build a rely/guarantee theory
- Justness can be imposed on termination and parallel composition

.


Robert J. Colvin, Ian J. Hayes, and Larissa A. Meinicke.
Designing a semantic model for a wide-spectrum language with concurrency.
Preliminary version at http://arxiv.org/abs/1609.00195, 2016.


I. J. Hayes.
Generalised rely-guarantee concurrency: An algebraic foundation.
Formal Aspects of Computing, 28(6):1057–1078, November 2016.


I.J. Hayes, R.J. Colvin, L.A. Meinicke, K. Winter, and A. Velykis.
An algebra of synchronous atomic steps.
In John Fitzgerald, Stefania Gnesi, and Constance Heitmeyer, editors, *Formal Methods 2016*, Lecture Notes in Computer Science. Springer Verlag, November 2016.
Accepted 9 August 2016.


I. J. Hayes, C. B. Jones, and R. J. Colvin.
Laws and semantics for rely-guarantee refinement.
Technical Report CS-TR-1425, Newcastle University, July 2014.